

pybmc: A Python package for Bayesian model combination

Troy Dasher^{1,2}, Kyle Godbey¹, and An Le^{1,2}

¹ Facility for Rare Isotope Beams, Michigan State University, East Lansing, MI 48824, USA

² Department of Physics and Astronomy, Michigan State University, East Lansing, MI 48824, USA

Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

750-1750 words <https://joss.readthedocs.io/en/latest/paper.html> <https://joss.readthedocs.io/en/latest/ex>

Summary

Bayesian model combination (BMC) provides a principled framework for combining predictions from multiple models while quantifying uncertainty. Unlike simple averaging methods, BMC explicitly accounts for correlations among models and infers optimal model weights through Bayesian inference, producing predictions with calibrated uncertainty estimates.

The pybmc package implements this methodology in Python, allowing researchers to apply Bayesian model combination to ensembles of predictive models defined on a shared domain. The package is designed for applications in which different theoretical or computational models produce systematically different predictions.

By providing a lightweight and flexible interface for constructing model ensembles and generating uncertainty-aware predictions, pybmc makes Bayesian model combination accessible for research workflows. While motivated by applications in nuclear physics, the approach is general and applicable to a broad range of problems involving ensemble modeling and uncertainty-aware prediction.

Statement of need - An

A section that clearly illustrates the research purpose of the software and places it in the context of related work. This should clearly state what problems the software is designed to solve, who the target audience is, and its relation to other work.

State of the fields

This package represents the first user-focused software implementation of the model orthogonalization and combination strategy detailed in [cite]. While there exist other Bayesian model mixing software, including the Taweret software package [cite Taweret JOSS], the current package is primarily designed to be used in scenarios where practitioners have precomputed databases of model predictions (with or without uncertainties) and wish to combine the results efficiently without needing to run additional simulations. This is particularly beneficial in nuclear physics, for instance, where global calculations of nuclear properties for all possible isotopes is computationally demanding.

Structure

An explanation of the trade-offs you weighed, the design/architecture you chose, and why it matters for your research application. This should demonstrate meaningful design thinking beyond a superficial code structure description.

pybmc is organized to provide a user-friendly, object-oriented API through the use of community tools while keeping the the user interface in Python. It is centered around two classes that separate data handling and the BMC workflow, along with helper functions to assist with the training and uncertainty quantification. This design isolates the two distinct functions that pybmc provides, clarifying the flow from preprocessed model outputs to final BMC estimates.

Data management is handled by the Dataset class, which provides utilities for loading data from external sources, selecting valid domains, and partitioning available observations into training, validation, and testing subsets. This abstraction centralizes data preparation within pybmc, ensuring that the inference workflow operates on consistent and well-defined inputs.

The BMC workflow is encapsulated in the BayesianModelCombination class, which serves as the main area for BMC. Rather than embedding certain functions directly in the class, supporting functionality such as sampling and uncertainty quantification is factored into helper modules. This design keeps the core workflow explicit and composable, emphasizes clarity in the training–prediction–uncertainty pipeline, and allows individual components to be modified or replaced without restructuring the overall package.

Research impact statement

The original methodology was published in a peer-reviewed journal [cite] and the first application of the methodology has also recently been published in an analysis of new experimental data from the Facility for Rare Isotope Beams [cite]. Once the package was completed, it began replacing the original, bespoke implementation in new scientific works including a study on Q-alpha trends in superheavy nuclei, the impact of nuclear uncertainties in r-process nucleosynthesis, and a systematic study of charge radii. Students involved in these projects have presented results obtained using the software at the American Physical Society's annual Division of Nuclear Physics meeting and at presentations locally. The package has also been adopted for release with the Bayesian Analysis for Nuclear Dynamics collaboration's V0.5 release and disseminated broadly to the nuclear physics community.

AI usage disclosure

Generative AI was used in the creation of a package template early in the development process and during development via inline autocomplete within the developers' editors. No code was adopted without human oversight and our metric for correctness was defined by detailed numerical comparison to the bespoke implementation already published in [cite]. AI tools were also used to expand the package documentation and write additional tests, with the resulting documentation going through a round of human review for completeness, correctness, and accessibility.

Acknowledgments

Citations